

# Clase del Módulo 3

Sitio: [Campus - Sadosky Capacitaciones](#)

Curso: Introducción al mundo de la programación paralela para HPC\_V1

Libro: Clase del Módulo 3

Imprimido por: Pablo Carrai

Día: miércoles, 10 de junio de 2026, 18:55

## Tabla de contenidos

- 1. Introducción al cómputo distribuido
- 2. Fundamentos de MPI: el "Hola mundo"
  - Compilación y ejecución: ejemplos
- 3. Comunicación punto a punto
  - El Peligro de deadlock
- 4. Comunicación colectiva
  - Recolección de datos y reducciones
- 5. Cierre

## 1. Introducción al cómputo distribuido

En la era del HPC y la *big data*, los problemas más complejos superan la capacidad de una sola computadora. La computación en clústeres, donde múltiples nodos (computadoras) trabajan en conjunto, es la norma.



A diferencia de OpenMP, que se especializa en memoria compartida (múltiples núcleos accediendo a la misma RAM), MPI (*Message Passing Interface*) es el estándar de facto para la programación en arquitecturas de memoria distribuida.

En este modelo, cada "proceso" tiene su propia memoria privada. La única forma de colaborar es enviando y recibiendo mensajes explícitos a través de una red.

En las siguientes secciones introduciremos los conceptos fundamentales de MPI y las dos formas por las que los procesos pueden comunicarse entre sí para coordinar la computación.

## 2. Fundamentos de MPI: el "Hola mundo"

### El Modelo SPMD

MPI utiliza predominantemente el modelo SPMD (*Single Program, Multiple Data*). Esto significa que lanza múltiples copias del mismo programa ejecutable. Cada copia es un proceso independiente.

Para que estos procesos colaboren, MPI les proporciona dos piezas clave de información:

Rank (ID): identificador único de cada proceso, similar al `omp_get_thread_num()`.

Size (NP): número total de procesos que participan en la computación.



Veamos un fragmento del siguiente video (minutos 3:40 a 9:36) que realiza una introducción didáctica a esta herramienta.



BIANCHINI, G. en Docencia Sistemas - UTN (21 de mayo de 2020). Computación Paralela: Herramientas para la programación paralela (Introducción a MPI). [Video]. YouTube.

### Funciones esenciales

MPI extiende los códigos programados en C/C++. Para ello se debe:

- Incluir el encabezado:

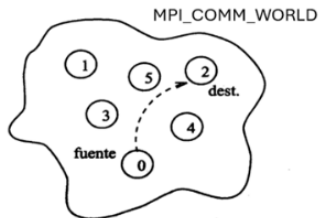
```
#include <mpi.h>
```

- Inicializar el entorno MPI:

```
MPI_Init(&argc, &argv);
```

- Finalizar el entorno MPI:

```
MPI_Finalize();
```



Dentro de este entorno, podemos consultar el rango (ID) y el tamaño (NP) total. El grupo de todos los procesos se llama comunicador, y el comunicador global por defecto es MPI\_COMM\_WORLD.

- `MPI_Comm_rank(MPI_COMM_WORLD, &rank)`  
(Obtiene el ID del proceso).
- `MPI_Comm_size(MPI_COMM_WORLD, &size)`  
(Obtiene el total de procesos).



Un comunicador es una colección de procesos, los cuales se comunican a través del envío y recepción de mensajes.

## Compilación y ejecución: ejemplos



En el siguiente video la profesora Verónica Gil Costa nos explica cómo escribir, compilar y ejecutar un programa en MPI.

Fundación Sadosky



Mirar en

Para compilar:



- `mpicc hola_mpi.c -o hola_mpi`
- `mpicxx hola_mpi.c -o hola_mpi`

Para ejecutar se usa un lanzador especial, comúnmente `mpirun`, o `mpiexec` para desplegar el programa en el clúster (o localmente para pruebas), especificando el número de procesos (`-np`).

- `mpirun -np 4 ./hola_mpi`

Veamos entonces el ejemplo 1: Hola Mundo MPI:

```
#include <stdio.h>

#include <mpi.h>

int main(int argc, char** argv) {

    int rank, size;

    // Inicializar el entorno MPI

    MPI_Init(&argc, &argv);

    // Obtener el rango (ID) y el tamaño (total de procesos)

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    MPI_Comm_size(MPI_COMM_WORLD, &size);

    //Cada proceso ejecuta este código

    printf("Hola desde el proceso %d de %d\n", rank, size);

    // Finalizar el entorno MPI

    MPI_Finalize();

    return 0;

}
```

El orden en que cada proceso ejecuta la instrucción `printf` no está garantizado, como puede verse al ejecutar el ejemplo con `mpirun -np 4 ./hola_mpi`.

- Hola desde el proceso 2 de 4
- Hola desde el proceso 0 de 4
- Hola desde el proceso 1 de 4
- Hola desde el proceso 3 de 4

Procesos *master-slave*

Para crear procesos *master-slave* (en castellano maestro-esclavo), se puede utilizar la siguiente estructura de código:

```
#include "mpi.h"

main(int argc, char,**argv)

{

    int nproc, mytid;

    MPI_Init(&argc,&argv);

    MPI_Comm_size(MPI_COMM_WORLD, &nproc);

    MPI_Comm_Rank(MPI_COMM_WORLD, &mytid);

    if (mytid ==0)

        master( );

    else

        slave( );

    MPI_Finalize();

}
```

#### Tipos de datos en MPI

La siguiente tabla muestra los tipos de datos utilizados por MPI:

| Identificador MPI  | Tipo de dato en C  |
|--------------------|--------------------|
| MPI_CHAR           | signed char        |
| MPI_SHORT          | signed short int   |
| MPI_INT            | signed int         |
| MPI_LONG           | signed long int    |
| MPI_UNSIGNED_CHAR  | unsigned char      |
| MPI_UNSIGNED_SHORT | unsigned short int |
| MPI_UNSIGNED       | unsigned int       |
| MPI_UNSIGNED_LONG  | unsigned long int  |
| MPI_FLOAT          | float              |
| MPI_DOUBLE         | double             |
| MPI_LONG_DOUBLE    | long double        |
| MPI_BYTE           | -                  |

### 3. Comunicación punto a punto

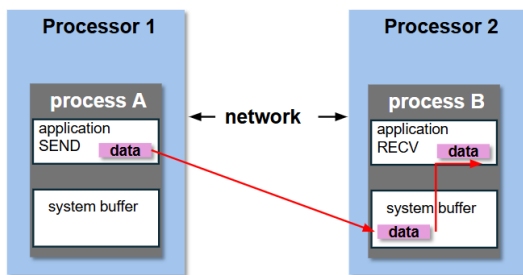
La comunicación punto a punto es la forma básica de comunicación: un proceso envía datos explícitamente a otro proceso. Lo hace del siguiente modo:

- El proceso origen envía un mensaje al proceso destino.
- El proceso destino recibe el mensaje.
- La comunicación se realiza dentro de un comunicador.
- El proceso destino se identifica por su rango dentro del comunicador.

#### Envío y recepción de mensajes bloqueantes

Las funciones básicas son bloqueantes:

#### MPI\_Send



La función no retorna hasta que el mensaje haya sido enviado (o, al menos, copiado a un *buffer* seguro).

`MPI_Send (void *buf, int count, MPI_Datatype datatype, int dest, int tag, MPI_Comm comm)`

En el siguiente ejemplo se envían 500 datos en punto flotante que se encuentran en la dirección *data* al proceso *\_dest* dentro del comunicador *MPI\_COMM\_WORLD* con la etiqueta *\_tag*.

```
MPI_Send (data, 500, MPI_FLOAT, _dest, _tag, MPI_COMM_WORLD)
```

#### MPI\_Recv

La función no retorna (se "bloquea") hasta que el mensaje esperado ha llegado.

`MPI_Recv (void *buf, int count, MPI_Datatype datatype, int source, int tag, MPI_Comm comm, MPI_Status *status)`

#### MPI\_Sendrecv

Esta función es una de las herramientas más útiles y seguras en MPI, diseñada específicamente para evitar el problema de *deadlock* (bloqueo mutuo) que ocurre comúnmente durante los intercambios de datos. Realiza un envío y una recepción en una sola llamada atómica.

En el siguiente ejemplo, recibe 500 datos en punto flotante y los almacena a partir de la dirección *value* de cualquier proceso dentro del comunicador *MPI\_COMM\_WORLD* con cualquier etiqueta.

```
MPI_Recv (value, 500, MPI_FLOAT, MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &status)
```

Por su parte, `MPI_sendrecv`:

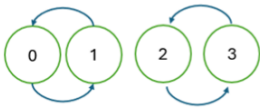
| Argumento / Variable               | Descripción                  |
|------------------------------------|------------------------------|
| --- Argumentos de ENVÍO ---        |                              |
| <code>void* sendbuf</code>         | Puntero a los datos a enviar |
| <code>int sendcount</code>         | Número de elementos a enviar |
| <code>MPI_Datatype sendtype</code> | Tipo de dato a enviar        |
| <code>int dest</code>              | Rango del proceso destino    |
| <code>int sendtag</code>           | Etiqueta del mensaje enviado |

| Argumento / Variable            | Descripción                                        |
|---------------------------------|----------------------------------------------------|
| --- Argumentos de RECEPCIÓN --- |                                                    |
| void* recvbuf                   | Puntero donde guardar los datos recibidos          |
| int recvcount                   | Máximo número de elementos a recibir               |
| MPI_Datatype recvtpe            | Tipo de dato a recibir                             |
| int source                      | Rango del proceso fuente                           |
| int recvtag                     | Etiqueta del mensaje a recibir                     |
| --- Contexto ---                |                                                    |
| MPI_Comm comm                   | Comunicador                                        |
| MPI_Status* status              | Puntero al estado (información sobre la recepción) |

## El Peligro de deadlock



Un *deadlock* o "abrazo mortal" ocurre cuando dos o más procesos se bloquean mutuamente, esperando cada uno que el otro actúe primero. Es el error más común en MPI.



Veamos en la figura de la izquierda un ejemplo de *deadlock*. Aquí, el Proceso 0 quiere enviar a 1, y el Proceso 1 quiere enviar a 0.

Veamos el ejemplo en detalle:

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char **argv) {
    int me, np, q, sendto;

    MPI_Status status;

    MPI_Init(&argc, &argv);

    MPI_Comm_size(MPI_COMM_WORLD, &np);

    MPI_Comm_rank(MPI_COMM_WORLD, &me);

    if (np%2==1) return 0;

    if (me%2==1) {sendto = me-1;}      // si es Proceso impar se comunicará con el par
anterior                               // por ej., si es proceso 1 se comunicará con 0

    else {sendto = me+1;}              // si es Proceso par se comunicará con el impar
siguiente

    // El proceso "me" espera recibir del procesador "sendto"

    MPI_Recv(&q, 1, MPI_INT, sendto, sendto, MPI_COMM_WORLD, &status);

    // Y luego enviar su ID al procesador "sendto"

    MPI_Send(&me, 1, MPI_INT, sendto, me, MPI_COMM_WORLD);

    printf("Sent %d to proc %d, received %d from proc %d\n", me, sendto, q, sendto);

    MPI_Finalize();

    return 0;
}
```

Sin embargo, al ejecutarse todos los procesos quedarán bloqueados antes de llegar enviar su ID vía la instrucción `MPI_Send`, pues están esperando recibir el ID de su proceso asociado que nunca llegarán a sus `MPI_Recv`.

Repasemos entonces algunas soluciones posibles:

Solución 1: ordenar las operaciones.



```

#include <mpi.h>
#include <stdio.h>
int main(int argc, char **argv) {
    int me, np, q, sendto;
    MPI_Status status;
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &np);
    MPI_Comm_rank(MPI_COMM_WORLD, &me);

    if (np%2==1) return 0;
    if (me%2==1) {sendto = me-1;}
    else {sendto = me+1;}

    if (me%2 == 0) {          // los procesos con ID par enviarán el mensaje
        MPI_Send(&me, 1, MPI_INT, sendto, me, MPI_COMM_WORLD);
        MPI_Recv(&q, 1, MPI_INT, sendto, sendto, MPI_COMM_WORLD, &status);
    } else {                  // los procesos con ID impar primero recibirán el
mensaje
        MPI_Recv(&q, 1, MPI_INT, sendto, sendto, MPI_COMM_WORLD, &status);
        MPI_Send(&me, 1, MPI_INT, sendto, me, MPI_COMM_WORLD);
    }
    printf("Sent %d to proc %d, received %d from proc %d\n", me, sendto, q, sendto);
    MPI_Finalize();
    return 0;
}

```

### Solución 2: utilizar MPI\_Sendrecv

```

#include <mpi.h>
#include <stdio.h>
int main(int argc, char **argv) {
    int me, np, q, partner;
    MPI_Status status; MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &np);
    MPI_Comm_rank(MPI_COMM_WORLD, &me);
    // Verificar que tengamos un número par de procesos
    if (np % 2 == 1) {
        if (me == 0) { printf("Este programa requiere un número par de procesos.\n"); }
        MPI_Finalize(); return 0; // Salir si el número de procesos es impar
    }
    // Determinar quién es la pareja de intercambio
    if (me % 2 == 1) {
        partner = me - 1; // Si soy impar, mi pareja es el anterior
    } else {
        partner = me + 1; // Si soy par, mi pareja es el siguiente
    }
    MPI_Sendrecv( &me, 1, MPI_INT, partner, me, // --- Parte de ENVÍO ---
                  &q, 1, MPI_INT, partner, partner, // --- Parte de RECEPCIÓN ---
                  MPI_COMM_WORLD, &status );
    // Imprimir el resultado
    printf("Proceso %d: Envié %d a %d, Recibí %d de %d\n", me, me, partner, q, partner);
    MPI_Finalize();
    return 0;
}

```

La salida será:

- Proceso 2: Envié 2 a 3, Recibí 3 de 3
- Proceso 3: Envié 3 a 2, Recibí 2 de 2
- Proceso 1: Envié 1 a 0, Recibí 0 de 0
- Proceso 0: Envié 0 a 1, Recibí 1 de 1

### Comunicación no bloqueante

Para evitar *deadlocks* y, más importante, para solapar cómputo con comunicación, MPI ofrece versiones no bloqueantes:

`MPI_Isend`: inicia un envío y retorna inmediatamente.

`MPI_Irecv`: inicia una recepción y retorna inmediatamente.

Estas funciones devuelven un objeto `MPI_Request`. Debemos verificar explícitamente cuándo la operación se completó usando:

```
MPI_Wait(MPI_Request* req, MPI_Status* status);
```

`MPI_Wait` toma como entrada un “recibo” (tipo de operación que quiere que se complete) y se bloquea hasta que la operación haya finalizado.

Por otro lado, la función `MPI_Test` verifica si una comunicación no bloqueante ha finalizado. El valor de la bandera (*flag*) indica si el mensaje ha sido transferido.

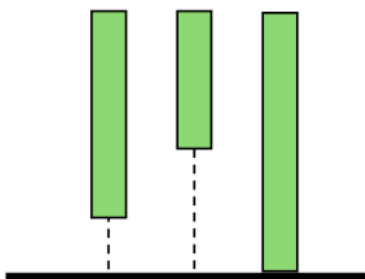
```
ierr = MPI_Test(request, flag, status);
```

## 4. Comunicación colectiva

### Barrera de sincronización

```
MPI_Barrier(MPI_COMM_WORLD);
```

Esta operación fuerza a todos los procesos a esperar en este punto hasta que todos hayan llegado.



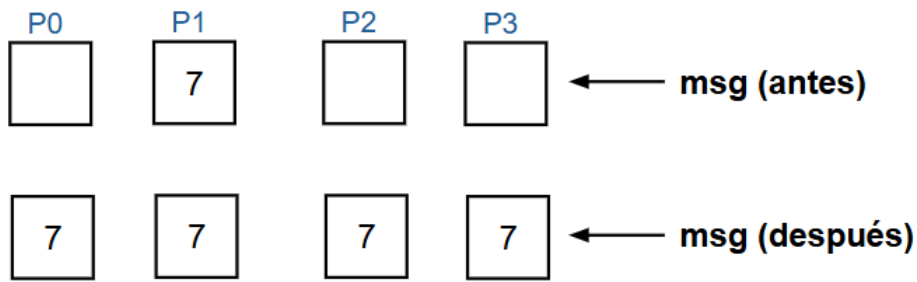
### Distribución de datos (*broadcast*, *scatter* y *alltoall*)

Broadcast (`MPI_Bcast`):

Uno-a-Todos. Los mismos datos se envían a todos los destinos. Todos los procesos ejecutan el *broadcast*, pero sólo el proceso que se indica como origen realiza el envío.

Envía un mensaje a todos los procesos del grupo:

```
MPI_Bcast (void *buffer, int count, MPI_Datatype datatype, int root, MPI_Comm comm)
```



Veamos a continuación un video que nos muestra paso a paso cómo escribir, compilar y ejecutar un código con MPI para hacer un *broadcast*:

Fundación Sadosky



Mirar en

```

#include <stdio.h>
#include <mpi.h>
#include <stdlib.h>
#include <string.h>
int main(int argc, char *argv[])
{
    int pid, nprocs;
    int max_len = 300;
    char texto[max_len];
    MPI_Init(&argc,&argv);
    MPI_Comm_size(MPI_COMM_WORLD,&nprocs);
    MPI_Comm_rank(MPI_COMM_WORLD,&pid);

    printf("pid %d\n",pid);
    if (pid==0)
    {
        strcpy(texto,"Hola desde MPI");
        printf("pid %d envia %s\n",pid,texto);
    }

    MPI_Bcast(texto,max_len,MPI_CHAR,0,MPI_COMM_WORLD);

    if(pid!=0)
    printf("pid %d texto %s\n",pid,texto);
    MPI_Finalize();
    return 0;
}

```

La salida será:

- pid 1
- pid 3
- pid 2
- pid 0
- pid 0 envía Hola desde MPI
- pid 1 envía Hola desde MPI
- pid 2 envía Hola desde MPI
- pid 3 envía Hola desde MPI

Ahora veremos una nueva operación llamada *scatter* (dispersión):

Scatter (MPI\_Scatter):

Uno-a-Todos (Distribuido). Un proceso root tiene un array grande y "reparte" trozos de ese array entre todos los procesos (incluyéndose a sí mismo).

Envía datos de un proceso a todos los procesos en el grupo. En el siguiente ejemplo, P1 contiene el dato a ser compartido:

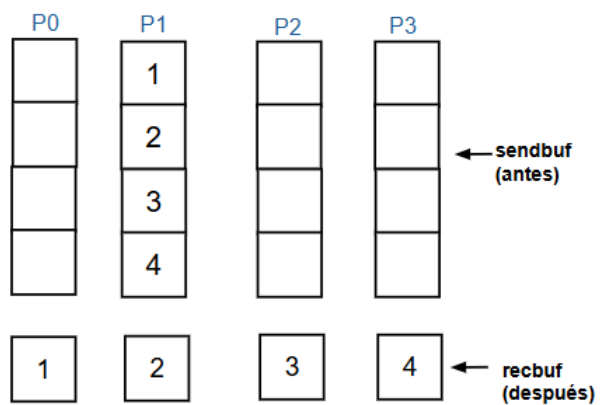
```
sendcnt = 1;
```

```
recvcnt = 1;
```

```

MPI_Scatter(sendbuf,sendcnt,MPI_INT,
            recvbuf,recvcnt,MPI_INT,
            src,MPI_COMM_WORLD);

```



El siguiente video muestra cómo utilizar diferentes funciones de MPI como *isend*, *irecv*, *broadcast* y *scatter*.

Fundación Sadosky



Mirar en

```

#include <stdio.h>
#include <mpi.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    int pid, nprocs,q;

    MPI_Init(&argc,&argv);
    MPI_Comm_size(MPI_COMM_WORLD,&nprocs);
    MPI_Comm_rank(MPI_COMM_WORLD,&pid);

    MPI_Status status;
    MPI_Request reqs;

    int dest;
    if (pid==nprocs-1) dest = 0;
    else dest = pid+1;

    MPI_Isend(&pid,1,MPI_INT,dest,1,MPI_COMM_WORLD,&reqs);
    MPI_Barrier(MPI_COMM_WORLD);
    MPI_Irecv(&q,1,MPI_INT,MPI_ANY_SOURCE,1,MPI_COMM_WORLD,&reqs);

    printf("pid %d recibe q=%d\n",pid,q);
    MPI_Barrier(MPI_COMM_WORLD);
    if (pid==0) printf("-----\n");
    if (pid==0)
        q=100;
    MPI_Bcast(&q,1,MPI_INT,0,MPI_COMM_WORLD);
    printf("pid %d q=%d\n",pid,q);
    MPI_Barrier(MPI_COMM_WORLD);
    if (pid==0) printf("-----\n");
    int N[20];
    if (pid==0)
        for(int i=0;i<20;i++) N[i]=i+1;

    int cant_rcv = 20/nprocs;
    printf("cant rec %d\n",cant_rcv);
    int data_recibida[cant_rcv];
    MPI_Scatter(N,cant_rcv,MPI_INT,data_recibida,cant_rcv,
               MPI_INT,0,MPI_COMM_WORLD);

    for(int i=0;i<cant_rcv;i++)
        printf("pid %d data_recibida[%d]=%d\n",pid,i,data_recibida[i]);

    MPI_Finalize();
    return 0;
}

```

La salida será:

- pid 1
- pid 3
- pid 2
- pid 0
- pid 0 envía Hola desde MPI
- pid 1 envía Hola desde MPI
- pid 2 envía Hola desde MPI
- pid 3 envía Hola desde MPI

**AlltoAll (MPI\_Alltoall):**

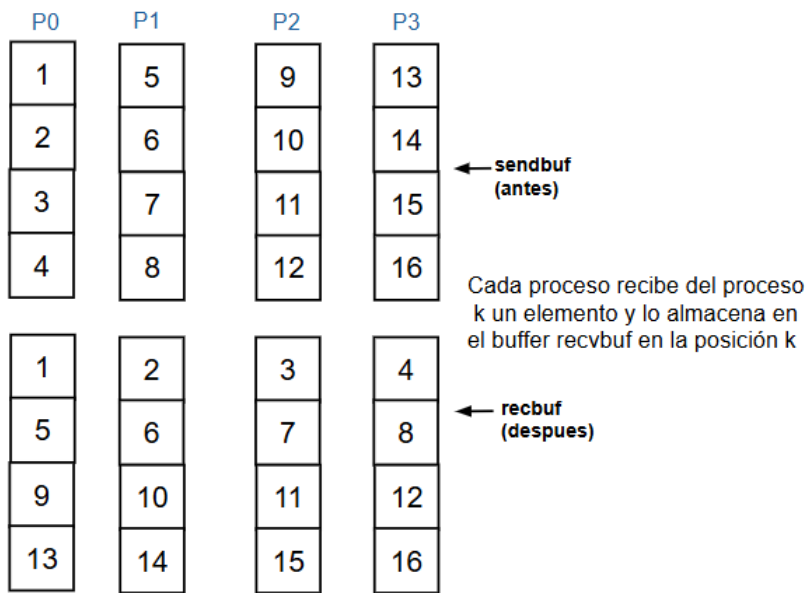
Envía un mensaje diferente de cada proceso a cada otro proceso.

Envía datos desde todos los procesos a todos los procesos en el grupo:

```
sendcnt = 1;
```

```
recvcnt = 1;
```

```
MPI_Alltoall(sendbuf, sendcnt, MPI_INT,
             recvbuf, recvcnt, MPI_INT,
             MPI_COMM_WORLD);
```

Veamos en un nuevo video cómo utilizar la función *alltoall*:

Fundación Sadosky



Mirar en

```
#include <stdio.h>
#include <mpi.h>
int main(int argc, char *argv[])
{
    int pid, nprocs;
    MPI_Init(&argc,&argv);
    MPI_Comm_size(MPI_COMM_WORLD,&nprocs);
    MPI_Comm_rank(MPI_COMM_WORLD,&pid);
    int A[nprocs], B[nprocs];
    for (int i=0;i<nprocs;i++)
        A[i]= pid*10;

    MPI_Alltoall(A,1,MPI_INT,B,1,MPI_INT,MPI_COMM_WORLD);

    for(int i=0;i<nprocs;i++)
        printf("pid %d rec[%d] %d\n",pid,i,B[i]);
    MPI_Finalize();
    return 0;
}
```

La salida será:

- pid 0 rec[0] = 0
- pid 0 rec[1] = 10
- pid 0 rec[2] = 20
- pid 0 rec[3] = 30
- pid 1 rec[0] = 0
- pid 1 rec[1] = 10
- pid 1 rec[2] = 20
- pid 1 rec[3] = 30
- pid 2 rec[0] = 0
- pid 2 rec[1] = 10
- pid 2 rec[2] = 20
- pid 2 rec[3] = 30
- pid 3 rec[0] = 0
- pid 3 rec[1] = 10
- pid 3 rec[2] = 20
- pid 3 rec[3] = 30



## Recolección de datos y reducciones

### Recolección de datos (*Gather*)

**Gather (MPI\_Gather):**

Todos-a-Uno. Es la operación inversa a *Scatter*. Cada proceso envía su resultado parcial al proceso *root*, que los ensambla en un array más grande. La recolección se realiza según el orden de rango en el comunicador.

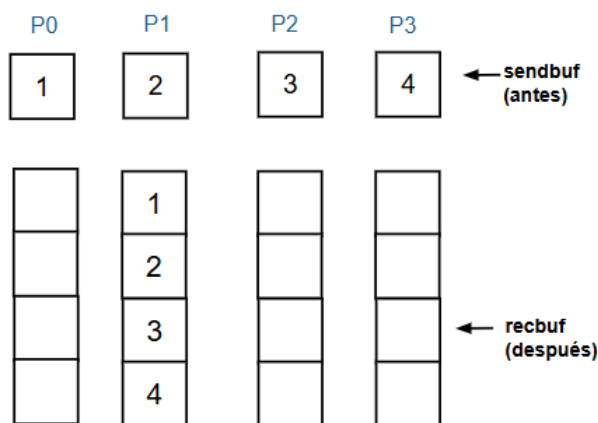
Recolecta los valores desde un grupo de procesos. Los mensajes serán recolectados en el P1:

```
sendcnt = 1;
```

```
recvcnt = 1;
```

```
src = 1;
```

```
MPI_Scatter(sendbuf, sendcnt, MPI_INT,  
            recvbuf, recvcnt, MPI_INT,  
            src, MPI_COMM_WORLD)
```



Ahora veamos un ejemplo de *gather* en video:

Fundación Sadosky



Mirar en

```
#include <stdio.h>

#include <mpi.h>

#include <stdlib.h>

#include <string.h>

int main(int argc, char *argv[])

{

    int pid, nprocs;

    MPI_Init(&argc,&argv);

    MPI_Comm_rank(MPI_COMM_WORLD,&pid);

    MPI_Comm_size(MPI_COMM_WORLD,&nprocs);

    int valor, *valores_recolectados;

    valor = pid*10;

    if (pid==0)

        valores_recolectados=(int*)malloc(nprocs*sizeof(int));

    MPI_Gather(&valor,1,MPI_INT,valores_recolectados,1,MPI_INT,0,MPI_COMM_WORLD);

    if (pid==0)

    {

        for (int i=0;i<nprocs;i++)

            printf("valor[%d]=%d\n",i,valores_recolectados[i]);

    }

    MPI_Finalize();

    return 0;

}
```

La salida será:

- valor [0]=0
- valor [1]=10
- valor [2]=20
- valor [3]=30

## Reducciones (*Reduce*)

Reduce (MPI\_Reduce):

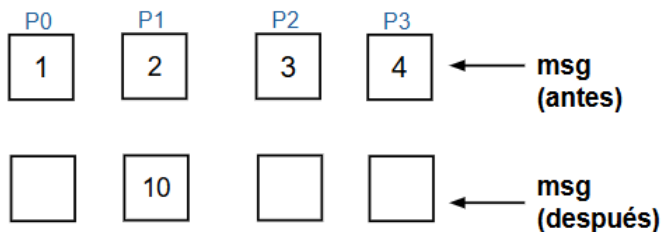
Todos-a-Uno (con operación). Cada proceso tiene un valor (ej. suma\_local). MPI recolecta todos esos valores, aplica una operación (ej. MPI\_SUM) y entrega el resultado final (ej. suma\_total) solo al proceso *root*.

Realiza una operación de reducción con todos los procesos en el grupo y coloca el resultado en solo un proceso.

```
count = 1;
```

```
dest = 1;
```

```
MPI_Reduce(sendbuf,recvbuf,count,MPI_INT,MPI_SUM, dest,MPI_COMM_WORLD);
```



| MPI Reduction Operation |                          | C Data Types               |
|-------------------------|--------------------------|----------------------------|
| MPI_MAX                 | Máximo                   | integer, float             |
| MPI_MIN                 | Mínimo                   | integer, float             |
| MPI_SUM                 | Suma                     | integer, float             |
| MPI_PROD                | Producto                 | integer, float             |
| MPI_LAND                | AND lógico               | integer                    |
| MPI_BAND                | AND a nivel de bit       | integer, MPI_BYTE          |
| MPI_LOR                 | OR lógico                | integer                    |
| MPI_BOR                 | OR a nivel de bit        | integer, MPI_BYTE          |
| MPI_LXOR                | XOR lógico               | Integer                    |
| MPI_BXOR                | XOR a nivel de bit       | integer, MPI_BYTE          |
| MPI_MAXLOC              | Máximo valor y ubicación | float, double, long double |
| MPI_MINLOC              | Mínimo valor y ubicación | float, double, long double |



A continuación veamos en video la explicación de cómo utilizar la función de MPI *reduce*:

Fundación Sadosky



Mirar en

```
#include <stdio.h>
#include <mpi.h>
#include <stdlib.h>
#include <string.h>
int main(int argc, char *argv[])
{
    int pid, nprocs;
    int valor, suma;

    MPI_Init(&argc,&argv);
    MPI_Comm_rank(MPI_COMM_WORLD,&pid);
    MPI_Comm_size(MPI_COMM_WORLD,&nprocs);

    valor = pid+100;
    MPI_Reduce(&valor,&suma,1,MPI_INT,MPI_SUM,0,MPI_COMM_WORLD);

    printf("pid %d suma=%d\n",pid,suma);
    MPI_Finalize();
    return 0;
}
```

La salida será:

- pid 1 suma=1707877936
- pid 3 suma=464625504
- pid 2 suma=-1972651008
- pid 0 suma=406

En este ejemplo, solo el procesador pid=0 tiene el valor correcto, debido a que es el procesador que recibe el resultado de la operación. Para ello reemplazar la función MPI\_Reduce por:

```
MPI_Allreduce(&valor,&suma_total,1,MPI_INT,MPI_SUM,MPI_COMM_WORLD);
```

La salida será:

- pid 0 max =406
- pid 1 max =406

- pid 2 max =406
- pid 3 max =406

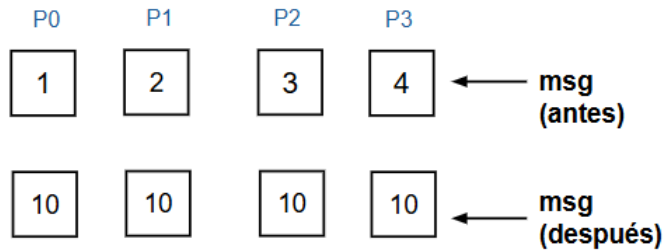
Allreduce (MPI\_Allreduce):

Es igual que *reduce*, pero el resultado final se entrega a todos los procesos, no solo al *root*.

Realiza una operación de reducción con todos los procesos en el grupo y coloca el resultado en todos los procesos:

```
count = 1;
```

```
MPI_Allreduce(sendbuf,recvbuf,count,MPI_INT,MPI_SUM, MPI_COMM_WORLD);
```



## 5. Cierre



¡Felicitaciones, llegaste hasta el final del módulo 3!

En este módulo exploramos los fundamentos esenciales de la programación en clústeres de computadoras mediante MPI, el estándar para el cómputo distribuido en HPC. Analizamos los mecanismos clave de comunicación punto a punto — bloqueantes y no bloqueantes—, los principales riesgos como el deadlock y sus estrategias de prevención, y las potentes operaciones colectivas que permiten distribuir y recolectar información de manera eficiente entre múltiples procesos.

Este módulo deja como mensaje central que MPI es una herramienta poderosa, flexible y esencial para abordar problemas que exceden la capacidad de una sola máquina, y que dominar sus mecanismos de comunicación es el primer paso para desarrollar aplicaciones paralelas robustas y eficientes en entornos HPC modernos.

En el próximo módulo nos adentraremos en la programación en GPU.



¡No te vayas sin practicar lo aprendido! Dirígete a la página de ejercitación haciendo [clic aquí](#).